

DAY 05

WEEK LECTURE COURSE - DAY 05

미들웨어 보안 필터 & 쿠키 세션 그리고 날씨 대시보드

서버 통신 최전선 권한 필터 및 SVG 차트 시각화

Target: 대학 1학년 및 취업훈련생 입문 과정

Framework: Next.js 14/15 + Tailwind CSS

Date: 2026 Curriculum Edition

1. 목차 (Table of Contents)

- 학습 목표
- 도입: 문지기 보안 요원 (Middleware)과 출입증 (Cookie)
- 개념 개론: Next.js Middleware의 역할과 생명주기
- 핵심 기술: 쿠키 기반의 로그인 검증 및 SVG 시각화
- 실습: Mini-Project 5 - 가상 로그인 및 날씨 대시보드 (`/dashboard` , `/login`)
- 실제 사례 연구: JWT 토큰 인증과 세션 하이재킹 방어 전략
- 결론 및 활용
- 강사 첨언 가이드 (Instructor Notes)

2. 학습 목표 (Learning Objectives)

1. **Next.js Middleware**의 동작 지점을 이해하고, 특정 URL 패턴에 대해 요청을 필터링하는 설정(`config`) 방법을 파악한다.
2. 클라이언트와 서버 간의 ****쿠키(Cookie)****를 통한 사용자 세션 인증 흐름을 설명할 수 있다.
3. 로그인하지 않은 사용자가 특정 경로에 접근하려 할 때, 미들웨어가 이를 즉시 감지하여 로그인 페이지로 리다이렉트(Route Protection)시키는 보안 코드를 설계한다.
4. 별도의 복잡한 라이브러리 없이, **순수 SVG 태그와 Tailwind CSS**만으로 미적인 데이터 차트를 그리는 기법을 학습한다.

3. 도입: 문지기 보안 요원 (Middleware)과 출입증 (Cookie)

지금까지 만든 웹페이지들은 인터넷 주소창에 주소만 치면 누구나 들어와서 볼 수 있었습니다. 하지만 사용자 개인의 소중한 정보가 담긴 대시보드나 관리자 페이지는 반드시 로그인을 거친 사람만 열어봐야 합니다.

💡 직관적인 비유로 이해하는 인증 보안 아키텍처

- 쿠키 (Cookie) - "클럽 입장용 야광 도장"
 - 입구에서 티켓(아이디/비밀번호)을 확인받으면 팔목에 야광 도장(Cookie)을 찍어줍니다. 이후부터는 다른 화장실, 바에 가거나 재입장할 때 티켓을 다시 보여줄 필요 없이 팔목 도장만 보여주면 됩니다. 브라우저는 서버로 요청을 보낼 때마다 이 도장을 자동으로 주머니에 넣어 같이 전송합니다.

- **미들웨어 (Middleware) - "입구에 서 있는 보안 요원"**

- 손님이 대시보드 전용 VIP 룸으로 들어가려고 문손잡이를 잡을 때, 문 바로 앞에 서 있는 보안 요원 (Middleware)이 가로막고 팔목 도장(Cookie)을 보여달라고 요청합니다. 도장이 없다면 안으로 들어갈 수 없게 막아서고 매표소(로그인 페이지)로 돌려보냅니다(리다이렉트).

4. 개념 개론: Next.js Middleware와 쿠키

Next.js의 미들웨어는 요청이 최종 목적지(페이지 렌더링)에 도달하기 전, 서버의 가장 최전선에서 실행되는 단일 파일입니다. 프로젝트 루트 폴더에 `middleware.ts` 파일을 생성해 두면 Next.js가 자동으로 모든 라우팅 전에 이 파일을 거쳐 가도록 작동시킵니다.

⚙️ 미들웨어 동작 원리 예시

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('session_token');

  if (!token) {
    // 토큰이 없다면 로그인 페이지로 즉시 돌려보냄
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // 토큰이 있다면 가던 길 가게 함
  return NextResponse.next();
}

// 미들웨어를 실행할 타겟 주소 지정
export const config = {
  matcher: ['/dashboard/:path*'],
};
```

5. 핵심 기술: SVG 차트 시각화

대시보드 데이터를 차트로 보여주기 위해 무거운 차트 라이브러리(Chart.js, Recharts 등)를 설치하고 배우려면 초급자에게는 며칠의 시간이 더 필요합니다. 우리는 HTML 표준인 ****SVG(Scalable Vector Graphics)****를

사용하여, 단 몇 줄의 직관적인 수치 매핑과 Tailwind CSS 효과를 통해 프리미엄 감성의 꺾은선 그래프를 직접 구축해 봅니다.

- `<svg>` : 벡터 그래픽 캔버스를 엽니다.
- `<polyline>` : 여러 점의 x, y 좌표 리스트를 주어 선을 이어 그립니다.
- `<circle>` : 그래프의 데이터 꼭짓점에 하이라이트 동그라미를 찍습니다.

6. 실습 - Mini-Project 5: 가상 로그인 및 날씨 대시보드

미들웨어 설정 및 쿠키 발급 API, 그리고 미려한 UI의 대시보드를 단일 웹 안에서 통합 구축합니다.

1) Middleware 생성: `middleware.ts`

프로젝트 루트 폴더(`C:/Users/User/.gemini/antigravity/scratch/nextjs-lecture-portal/middleware.ts`)에 생성합니다.

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const session = request.cookies.get('antigravity_session');
  const { pathname } = request.nextUrl;

  // 대시보드 경로로 접근하는데 쿠키가 없는 경우 → 로그인으로 리다이렉트
  if (pathname.startsWith('/dashboard') && !session) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // 로그인 페이지로 오는데 이미 쿠키(로그인 정보)가 있는 경우 → 대시보드로 자동 연결
  if (pathname.startsWith('/login') && session) {
    return NextResponse.redirect(new URL('/dashboard', request.url));
  }

  return NextResponse.next();
}

// 미들웨어가 작동할 경로 정의
export const config = {
  matcher: ['/dashboard/:path*', '/login'],
};
```

2) 가상 로그인 API 생성: `app/api/auth/login/route.ts`

사용자 로그인 성공 시 쿠키를 생성해 내려주는 API 라우트입니다.

```
import { NextResponse } from 'next/server';

export async function POST(request: Request) {
  try {
    const { username, password } = await request.json();

    // 데모용 가상 계정 검증 (실무에서는 데이터베이스 확인)
    if (username === 'admin' && password === 'admin123') {
      const response = NextResponse.json({ success: true, message: '로그인 성공!' });

      // 세션 쿠키 발급 (HttpOnly 옵션으로 보안 강화)
      response.cookies.set('antigravity_session', 'demo-session-token-999', {
        httpOnly: true,
        path: '/',
        maxAge: 60 * 30, // 30분 유효
        sameSite: 'lax',
      });

      return response;
    }

    return NextResponse.json(
      { success: false, message: '아이디(admin) 또는 비밀번호(admin123)가 맞지 않습니다.' },
      { status: 401 }
    );
  } catch (e) {
    return NextResponse.json({ success: false, message: '서버 에러가 발생했습니다.' }, { status: 500 });
  }
}
```

3) 가상 로그아웃 API 생성: `app/api/auth/logout/route.ts`

쿠키를 만료시켜 로그아웃시키는 API 라우트입니다.

```
import { NextResponse } from 'next/server';

export async function POST() {
  const response = NextResponse.json({ success: true, message: '로그아웃 되었습니다' });

  // 쿠키를 만료(MaxAge = 0)시켜 브라우저에서 제거
  response.cookies.set('antigravity_session', '', {
    path: '/',
    maxAge: 0,
  });

  return response;
}
```

4) 로그인 페이지 생성: `app/login/page.tsx`

아이디 `admin` , 비밀번호 `admin123` 으로 로그인을 시도하는 깔끔한 로그인 폼입니다.

```

'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

export default function LoginPage() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [errorMsg, setErrorMsg] = useState('');
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMsg('');
    setLoading(true);

    try {
      const res = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ username, password }),
      });

      const data = await res.json();

      if (data.success) {
        // 성공 시 대시보드로 이동 (미들웨어가 이제 접근을 허용함)
        router.refresh(); // 미들웨어 쿠키 인식 갱신
        router.push('/dashboard');
      } else {
        setErrorMsg(data.message);
      }
    } catch (err) {
      setErrorMsg('로그인 통신 중 오류가 발생했습니다.');
```

```

    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100 flex flex-col justify

```



```

<div className="max-w-md w-full bg-slate-900 border border-slate-800 rounded

  { /* 상단 텍스트 */ }
  <div className="text-center space-y-2">
    <Link href="/" className="text-xs text-teal-400 hover:underline font-semibold">
      &larr; 홈으로 돌아가기
    </Link>
    <h1 className="text-3xl font-extrabold tracking-tight bg-gradient-to-r from-teal-400 to-teal-600">
      Service Login
    </h1>
    <p className="text-xs text-slate-500">
      대시보드 입장을 위한 관리자 계정을 입력해 주세요.
    </p>
  </div>

  { /* 안내 배너 */ }
  <div className="bg-slate-950/80 border border-slate-800/40 p-4 rounded-xl">
    <p className="font-bold text-teal-400">💡 학습용 데모 계정 정보</p>
    <p className="text-slate-400">아이디: <code className="text-slate-200">admin</code>
    <p className="text-slate-400">비밀번호: <code className="text-slate-200">123456</code>
  </div>

  { /* 에러 피드백 */ }
  {errorMsg && (
    <div className="bg-red-950/60 border border-red-800/30 text-red-400 px-4 py-3">
      ⚠️ {errorMsg}
    </div>
  )}

  { /* 로그인 폼 */ }
  <form onSubmit={handleLogin} className="space-y-4">
    <div className="space-y-1.5">
      <label className="text-xs font-semibold text-slate-400">Username</label>
      <input
        type="text"
        required
        value={username}
        onChange={(e) => setUsername(e.target.value)}
        placeholder="admin"
        className="w-full bg-slate-950 border border-slate-800 rounded-xl px-3 py-2"
      />
    </div>

    <div className="space-y-1.5">

```

```

        <label className="text-xs font-semibold text-slate-400">Password</label>
        <input
          type="password"
          required
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="....."
          className="w-full bg-slate-950 border border-slate-800 rounded-xl p-3"
        />
      </div>

      <button
        type="submit"
        disabled={loading}
        className="w-full py-3 bg-teal-500 text-slate-950 font-bold rounded-xl"
      >
        {loading ? '로그인 확인 중...' : '대시보드 로그인'}
      </button>
    </form>

  </div>
</div>
);
}

```

5) 날씨 대시보드 페이지 생성: `app/dashboard/page.tsx`

쿠키 검증이 통과된 사용자만 들어와서 볼 수 있는 실시간 날씨 데이터 시각화 대시보드입니다.

```

'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

export default function DashboardPage() {
  const router = useRouter();
  const [loggingOut, setLoggingOut] = useState(false);

  // 시뮬레이션용 주간 날씨 기온 추이 데이터
  const weeklyTemps = [21, 23, 22, 26, 28, 25, 24];
  const days = ['월', '화', '수', '목', '금', '토', '일'];

  // 로그아웃 처리 함수
  const handleLogout = async () => {
    setLoggingOut(true);
    try {
      const res = await fetch('/api/auth/logout', { method: 'POST' });
      if (res.ok) {
        router.refresh(); // 미들웨어 쿠키 인식 해제 유도
        router.push('/login');
      }
    } catch (e) {
      console.error(e);
    } finally {
      setLoggingOut(false);
    }
  };

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100 selection:bg-teal-500">
      <div className="max-w-5xl mx-auto space-y-8">

        {/* 상단 네비게이션 헤더 */}
        <div className="flex justify-between items-center border-b border-slate-900">
          <div className="space-y-1">
            <span className="text-xs uppercase tracking-widest text-teal-400 font-mono">
              Authorized Session
            </span>
            <h1 className="text-3xl font-extrabold tracking-tight bg-gradient-to-r from-teal-400 to-slate-100">
              Weather & Operations Dashboard
            </h1>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

    </div>
    <button
      onClick={handleLogout}
      disabled={loggingOut}
      className="px-4 py-2 bg-red-950 text-red-400 border border-red-800/30
    >
      {loggingOut ? '종료 중...' : '로그아웃'}
    </button>
  </div>

  {/* 3열 데이터 카드 레이아웃 */}
  <div className="grid md:grid-cols-3 gap-6">
    <div className="bg-slate-900 border border-slate-800 p-6 rounded-2xl spa
      <span className="text-xs text-slate-500 font-medium">현재 기온 (서울)</span>
      <h3 className="text-4xl font-extrabold text-teal-400">24.5 °C</h3>
      <p className="text-xs text-slate-400 pt-2">어제 평균 대비 +1.2 °C 상승</p>
    </div>
    <div className="bg-slate-900 border border-slate-800 p-6 rounded-2xl spa
      <span className="text-xs text-slate-500 font-medium">대기 습도</span>
      <h3 className="text-4xl font-extrabold text-emerald-400">62 %</h3>
      <p className="text-xs text-slate-400 pt-2">야외 쾌적 수준 유지 중</p>
    </div>
    <div className="bg-slate-900 border border-slate-800 p-6 rounded-2xl spa
      <span className="text-xs text-slate-500 font-medium">평균 풍속</span>
      <h3 className="text-4xl font-extrabold text-sky-400">3.4 m/s</h3>
      <p className="text-xs text-slate-400 pt-2">남남서풍 불어오는 상태</p>
    </div>
  </div>

  {/* 주간 기온 변화 차트 영역 (SVG 수동 매핑 꺾은선 차트) */}
  <div className="bg-slate-900 border border-slate-800 p-6 rounded-3xl space
    <div className="flex justify-between items-center">
      <h4 className="font-bold text-lg">주간 기온 변화 추이 (Weekly Temp Curv
      <span className="text-xs text-slate-500">단위: °C</span>
    </div>

    {/* SVG 선 차트 */}
    <div className="relative h-64 bg-slate-950 rounded-2xl border border-sla
      <div className="w-full h-40 flex items-end justify-between relative mt

        {/* 뒷배경 가로 점선 가이드라인 */}
        <div className="absolute inset-0 flex flex-col justify-between point
          <div className="border-b border-slate-900 w-full" />
          <div className="border-b border-slate-900 w-full" />

```

```

    <div className="border-b border-slate-900 w-full" />
  </div>

  { /* SVG 폴리라인 차트 */ }
  <svg className="absolute inset-0 w-full h-full" preserveAspectRatio=
    { /* 꺾은선 (Polyline) */ }
    <polyline
      fill="none"
      stroke="#14b8a6"
      strokeWidth="3.5"
      strokeLinecap="round"
      strokeLinejoin="round"
      points="50,110 150,90 250,100 350,60 450,40 550,70 650,80"
      className="drop-shadow-[0_4px_12px_rgba(20,184,166,0.3)]"
    />

  { /* 꼭짓점 동그라미 매핑 */ }
  {[
    {x: 50, y: 110, temp: 21},
    {x: 150, y: 90, temp: 23},
    {x: 250, y: 100, temp: 22},
    {x: 350, y: 60, temp: 26},
    {x: 450, y: 40, temp: 28},
    {x: 550, y: 70, temp: 25},
    {x: 650, y: 80, temp: 24}
  ]}.map((pt, idx) => (
    <g key={idx} className="group/dot">
      <circle
        cx={pt.x}
        cy={pt.y}
        r="6"
        fill="#14b8a6"
        stroke="#020617"
        strokeWidth="2"
      />
      <text
        x={pt.x}
        y={pt.y - 12}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="10"
        className="font-bold"
      >
        {pt.temp}°

```

```

        </text>
      </g>
    )})
  </svg>

</div>

  { /* X축 요일 표시 */ }
  <div className="flex justify-between text-xs text-slate-500 font-semib">
    {days.map((day, idx) => (
      <span key={idx} className="w-12 text-center">{day}요일</span>
    ))}
  </div>
</div>
</div>

</div>
</div>
);
}

```

7. 실제 사례 연구: JWT 토큰 인증과 보안 세션

쿠키 기반 로그인 설계는 아주 직관적이지만, 실제 대형 서비스에서는 단순 세션 아이디 대신 암호화된 ****JWT(JSON Web Token)****를 발급받아 쿠키에 저장하는 방식을 많이 씁니다.

- **보안 꿀팁:** 쿠키 발급 시 `httpOnly: true` 옵션을 적용하면 해커가 자바스크립트의 `document.cookie` 명령어로 쿠키를 읽어가는 것(XSS 공격)을 브라우저 보안 레벨에서 완전히 차단하므로 매우 안전합니다. 우리가 작성한 API 코드에도 이 보안 옵션이 적용되어 있습니다.

8. 결론 및 활용

- **요약:** Next.js Middleware는 사용자 요청의 문지기 역할을 하여 강력한 Route Protection을 수행합니다. 쿠키와 미들웨어를 함께 사용하면 유려하고 단단한 인증 웹 서비스를 완성할 수 있습니다.
- **숙제:** 로그인 버튼 작동 시 메인 홈(/)으로 넘어가도록 소스코드를 조금 수정하고, 로그인하지 않고 `/dashboard` 주소로 갔을 때 정말로 로그인으로 튕겨 나가는지 테스트해보세요.

9. 강사 첨언 가이드 (Instructor Notes)

💡 TIP

강사 강의 꿀팁

- **00m ~ 30m:** 쿠키가 통신 시마다 HTTP 요청 헤더에 자동으로 탑재된다는 개념을 브라우저 개발자 도구의 Application 탭을 켜서 직접 보여주며 강의하십시오.
- **30m ~ 50m:** 미들웨어가 서버 전송 단에서 매칭 경로 패턴에 따라 가로채는 필터 흐름도를 칠판에 시각화하여 그리세요.
- **50m ~ 120m:** 5일차 최종 대시보드를 구축하게 한 후, 직접 `/dashboard` 주소를 복사해 새 시크릿 창에서 테스트하게 시연하여 보안 라우트 보호 기능을 체험하게 해줍니다.

⚠️ WARNING

자주 발생하는 에러 및 대처법

1. **쿠키 헤더 미반영에 따른 무한 루프:** 미들웨어가 쿠키를 확인하지 않고 잘못 리다이렉트하거나 `/login` 페이지 마저 문지기가 차단하면 무한 리다이렉트 루프에 빠져 브라우저가 정지합니다. 반드시 `config.matcher` 에서 `/login` 과 `public` 정적 자산 경로들을 잘 제외하거나 미들웨어 내부 조건식으로 차단 필터를 잘 분기했는지 코드를 확인해 주세요.
2. **쿠키 적용 후 새로고침 안 됨:** 브라우저에서 `NextResponse.cookies` 를 셋팅한 후 가끔 라우터가 즉각 상태를 인지하지 못할 때가 있습니다. 반드시 클라이언트 로직 단에서 `router.refresh()` 를 수행하여 브라우저 컴포넌트 세션 상태를 동기화해 주도록 가이드하세요.